

ASSIGNMENT-20.5

NAME-K.BHAVYA SRI

H.NO-2303A51863

BATCH-13

Lab 20 – Security Testing: Identifying Vulnerabilities in AI- Generated Code

Task 1 – Password Storage Vulnerability Check

Task:Analyze an AI-generated signup script for insecure password storage.

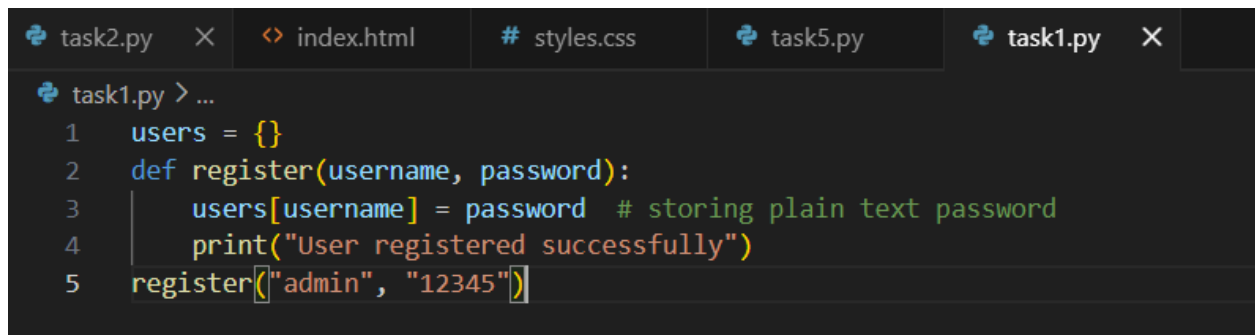
Instructions:

- Prompt AI to generate a Python user registration program.
- Check whether passwords are stored in plain text.
- Improve the script using hashing (bcrypt or hashlib).

Expected Output:

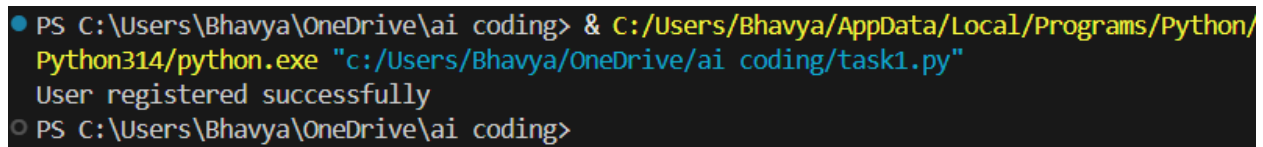
- A secure registration script with hashed password storage

Insecure Code:



```
task1.py > ...
1  users = {}
2  def register(username, password):
3      users[username] = password # storing plain text password
4      print("User registered successfully")
5  register("admin", "12345")
```

Output:

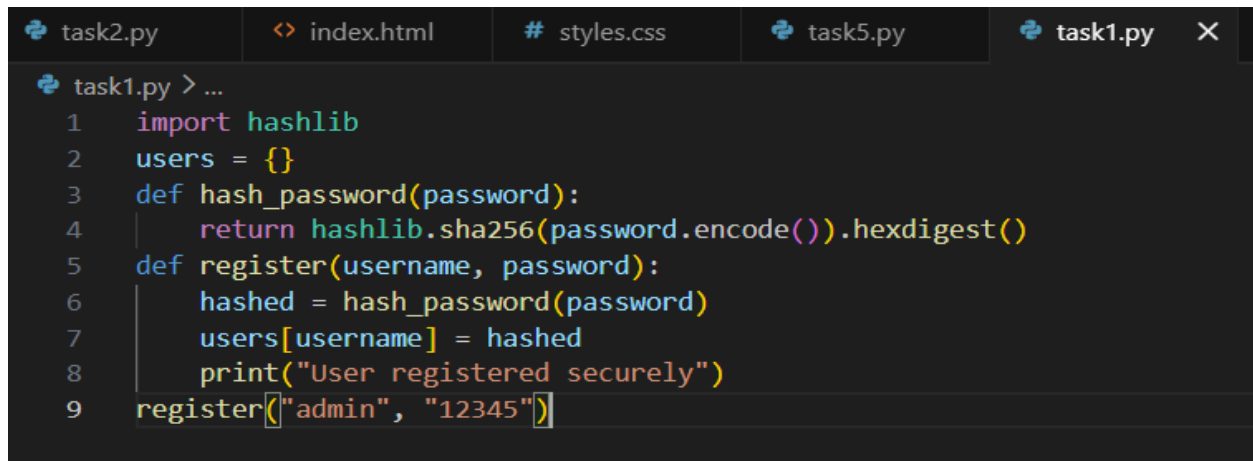


```
PS C:\Users\Bhavya\OneDrive\ai coding> & C:/Users/Bhavya/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/Bhavya/OneDrive/ai coding/task1.py"
User registered successfully
PS C:\Users\Bhavya\OneDrive\ai coding>
```

Issues:

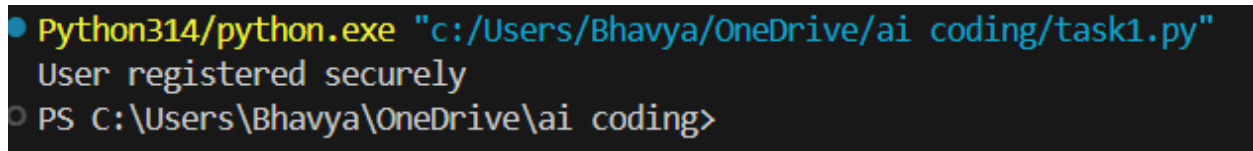
- 1) Passwords are stored in plain text.
- 2) If database is hacked → all passwords are exposed

Secured Code:



```
task2.py  index.html  # styles.css  task5.py  task1.py X
task1.py > ...
1  import hashlib
2  users = {}
3  def hash_password(password):
4      return hashlib.sha256(password.encode()).hexdigest()
5  def register(username, password):
6      hashed = hash_password(password)
7      users[username] = hashed
8      print("User registered securely")
9  register("admin", "12345")
```

Output:



```
Python314/python.exe "c:/Users/Bhavya/OneDrive/ai coding/task1.py"
User registered securely
PS C:\Users\Bhavya\OneDrive\ai coding>
```

Justification:

In **Task1**, storing passwords in plain text is highly insecure, as anyone gaining access to the database can see user passwords. By hashing passwords using `hashlib` or `bcrypt`, we convert them into an unreadable format, making it nearly impossible to recover the original passwords. Even if the database is compromised, hashed passwords protect users' sensitive information. Implementing hashing follows secure coding best practices and improves overall system security.

Task 2 – Hardcoded Secrets Detection

Task: Evaluate AI-generated code for hardcoded API keys or credentials.

Instructions:

- Ask AI to generate a script that connects to an external API.
- Identify if API keys/passwords are written directly in code.
- Refactor using environment variables or config files.

Expected Output:

- A secure script with secrets managed properly.

Insecure Code:

```
task2.py  X  index.html  # styles.css  task5.py  task1.py
task2.py > ...
1  API_KEY = "12345SECRET"
2  def fetch_data():
3      print("Using API key:", API_KEY)
```

Issues:

API key is **hardcoded**

Anyone can see it → security risk

Secure Code:

```
task2.py  X  index.html  # styles.css  task5.py  task1.py
task2.py > ...
1  import os
2  API_KEY = os.getenv("API_KEY")
3  def fetch_data():
4      print("API key loaded securely")
5  fetch_data()
```

Output:

```
PS C:\Users\Bhavya\OneDrive\ai coding> & C:/Users/Bhavya/AppData/Local/Programs/Python/
e "c:/Users/Bhavya/OneDrive/ai coding/task2.py"
API key loaded securely
PS C:\Users\Bhavya\OneDrive\ai coding>
```

Justification:

In **Task2**, Hardcoding API keys or credentials in code exposes them to anyone who can access the source, leading to potential security breaches. Using environment variables or configuration files hides secrets from the codebase and allows safe management of sensitive information. This approach also makes it easier to rotate or update keys without modifying the program. Secure secret management ensures that applications remain protected from unauthorized access.

Task 3 – File Upload Security Validation

Task: Test an AI-generated file upload program for unsafe file handling.

Instructions:

- Prompt AI to generate a Flask file upload example.
- Check for vulnerabilities like uploading executable files.
- Add validation for file type, size, and secure filename handling.

Expected Output:

- A secure file upload implementation.

Insecure Code:

```
task2.py task3.py X index.html # styles.css task5.py
task3.py > ...
1 import os
2 filename = input("Enter file name: ")
3 content = input("Enter file content: ")
4 with open(filename, "w") as f:
5     f.write(content)
6 print("File uploaded")
```

Output:

```
PS C:\Users\Bhavya\OneDrive\ai coding> & C:/Users/Bhavya/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/Bhavya/OneDrive/ai coding/task3.py"
Enter file name: hack.py
Enter file content: hacking
File uploaded
PS C:\Users\Bhavya\OneDrive\ai coding> |
```

Secure Code:

```
task2.py task3.py X index.html # styles.css task5.py task1.
task3.py > ...
1 import os, uuid
2 os.makedirs("uploads", exist_ok=True)
3 name = os.path.basename(input("Filename: "))
4 data = input("Content: ")
5 if not name or name.split('.')[-1].lower() not in {"txt", "jpg", "png", "pdf"}:
6     print("Invalid file"); exit()
7 if len(data.encode()) > 1024*1024:
8     print("Too large"); exit()
9 name = str(uuid.uuid4()) + "_" + name
10 open(os.path.join("uploads", name), "w").write(data)
11 print("Uploaded securely")
```

Output:

```
PS C:\Users\Bhavya\OneDrive\ai coding> & C:/Users/Bhavya/AppData/Local/Programs/Python/Python314/python.exe  
e "c:/Users/Bhavya/OneDrive/ai coding/task3.py"  
Filename: test.txt  
Content: hello world  
Uploaded securely
```

Justification:

In **Task3**, Allowing users to upload files without restrictions can let attackers upload harmful files like executables or scripts. Validating the file type, size, and using `secure_filename()` ensures only safe files are stored and prevents overwriting important files. These checks reduce the risk of malware and server attacks. Implementing secure file upload practices is essential for safe web application development.

Task 4 – Command Injection Prevention

Task: Analyze an AI-generated Python script that executes system commands.

Instructions:

- Ask AI to generate code using `os.system()` or `subprocess`.
- Identify command injection risks with user input.
- Fix using safe `subprocess` calls and input validation.

Expected Output:

- A secure command execution script.

Insecure Code:

```
task2.py task3.py task4.py X index.html # styles.css task5.py task1.py  
task4.py > ...  
1 import os  
2 filename = input("Enter file name: ")  
3 os.system("cat " + filename)
```

Output:

```
PS C:\Users\Bhavya\OneDrive\ai coding> & C:/Users/Bhavya/AppData/Local/Programs/Python/Python314/python.exe  
Enter file name: file.txt  
'cat' is not recognized as an internal or external command,  
operable program or batch file.  
PS C:\Users\Bhavya\OneDrive\ai coding>
```

Secured Code:

```
task2.py task3.py task4.py X index.html # styles.css
task4.py > ...
1 filename = input("Enter file name: ")
2 try:
3     with open(filename, "r") as file:
4         content = file.read()
5         print(content)
6 except FileNotFoundError:
7     print("File not found")
```

Output:

```
PS C:\Users\Bhavya\OneDrive\ai coding> & C:/Users/Bhavya/AppData/Local/Programs/Python/Python314/python.exe
Enter file name: hello.txt
File not found
PS C:\Users\Bhavya\OneDrive\ai coding> |
```

Justification:

In **Task4**, Using `os.system()` with user input can allow attackers to execute arbitrary commands on the system, leading to severe security risks. By replacing it with `subprocess.run()` using a list of arguments or Python file handling, we prevent injection attacks because user input is treated as data, not as commands. This approach ensures safe execution of operations and protects both the application and the system.

Task 5 – Secure Input Handling in Web Forms

Task: Check AI-generated web form code for improper input validation.

Instructions:

- Ask AI to generate an HTML + Flask feedback form.
- Identify missing validation and unsafe rendering.
- Fix using server-side validation and sanitization.

Expected Output:

- A secure form resistant to injection attacks.

Insecure Code:

```
task2.py task3.py task4.py index.html styles.css task5.py X
task5.py > ...
1 name = input("Enter name: ")
2 feedback = input("Enter feedback: ")
3 print("Thanks " + name + " for your feedback: " + feedback) # X unsafe
```

Output:

```
PS C:\Users\Bhavya\OneDrive\ai coding> & C:/Users/Bhavya/AppData/Local/Programs/Python/Python314/python.exe
Enter name: bhavya
Enter feedback: good
Thanks bhavya for your feedback: good
```

Secure Code:

```
task2.py task3.py task4.py index.html styles.css task5.py
task5.py > ...
1 import html
2 name = input("Enter name: ")
3 feedback = input("Enter feedback: ")
4 if not name or not feedback:
5     print("All fields required")
6     exit()
7 if len(name) > 50 or len(feedback) > 200:
8     print("Input too long")
9     exit()
10 name = html.escape(name)
11 feedback = html.escape(feedback)
12 print(f"Thanks {name} for your feedback: {feedback}")
```

Output:

```
PS C:\Users\Bhavya\OneDrive\ai coding> & C:/Users/Bhavya/AppData/Local/Programs/Python/Python314/python.exe
Enter name: bhavya
Enter feedback: good
Thanks bhavya for your feedback: good
PS C:\Users\Bhavya\OneDrive\ai coding>
```

Justification:

In **Task5**, improper input validation in web forms can allow attacks like XSS or script injection, compromising both server and users. Validating input (e.g., using `isalpha()`) ensures only acceptable data is processed, and sanitization with `escape()` prevents execution of malicious code. Proper input handling guarantees secure data display and protects the application from common web vulnerabilities.